// DEVELOPER WORKSHOP

Everybody hates monorepos

Building a solid Python monorepo with uv

DERVIS VAN LEERSUM · SENIOR PYTHON ENGINEER

dervis.dev

// WHOAMI

Hi, I'm Dervis



Senior Python Engineer

Sopra Steria · Utrecht

dervis@grid: ~

```
$ whoami
senior python engineer · energy grid
$ cat origin.txt
civil engineer turned software engineer
$ cat community.txt
treasurer @ PyCon NL / PyNetherlands
co-founder @ PyNetherlands
$ ls teaching/
4-day-python-course  hbo  workshops
$ _
```

// THE DEAL

You, 80 minutes from now

01 · ✅

A working uv workspace on your laptop

02 · ✅

An understanding of common traps and how to avoid them 🔧

03 · ✅

A `.just/` toolkit ready to use

04 · ✅

A clear answer to “should we actually do this?”

// MISSION BRIEFING

What's on the program

0 SETUP
Bootstrap

1 HISTORY
Act I-IV

2 WORKSHOP
Movement 1-3

3 FINALE
Ship it

Fin WRAP-UP
Questions and next steps

00

// SETUP

Bootstrap

Start the download. I'll talk while it runs.

// SETUP

Bootstrap

REPOSITORY



Scan to clone

DISCORD



Scan to join

setup

```
$ git clone dervis.dev/pizza-monorepo-wrkshp
$ cd pizza-monorepo-wrkshp
$ uv tool install rust-just
$ just setup
```

Green? You're good!

Red? Something is wrong

Windows? Use WSL or Git Bash, not PowerShell

Slow? The first sync is the big download

01

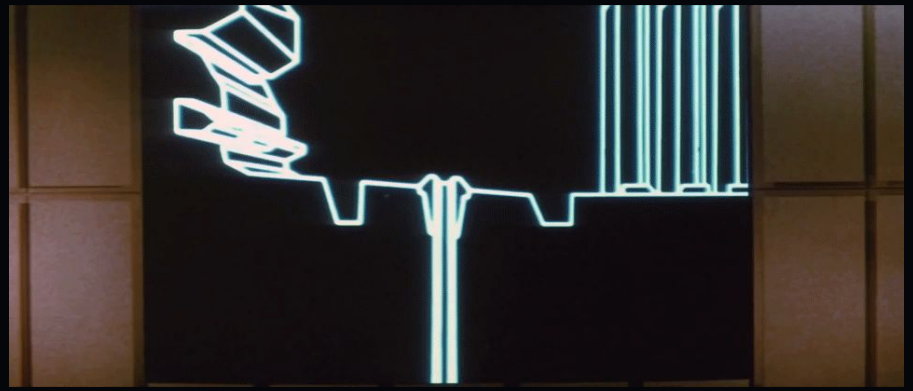
// ACT I-IV

History

The Monosith, The Great Repo Scattering, The Monorepo Strikes Back, and Meanwhile in Python

One deployable, one blast radius

- Blast radius of a typo: the entire company



The Death Star of codebases.
One thermal exhaust port💥

And everyone knew it was there.
Just not exactly where.

The hidden tax nobody invoices

- Shared code means packaging, versioning and releasing five times
- One change means two PRs, two reviews, and a prayer they merge in order
- *“Which version is live?”* a mystery!

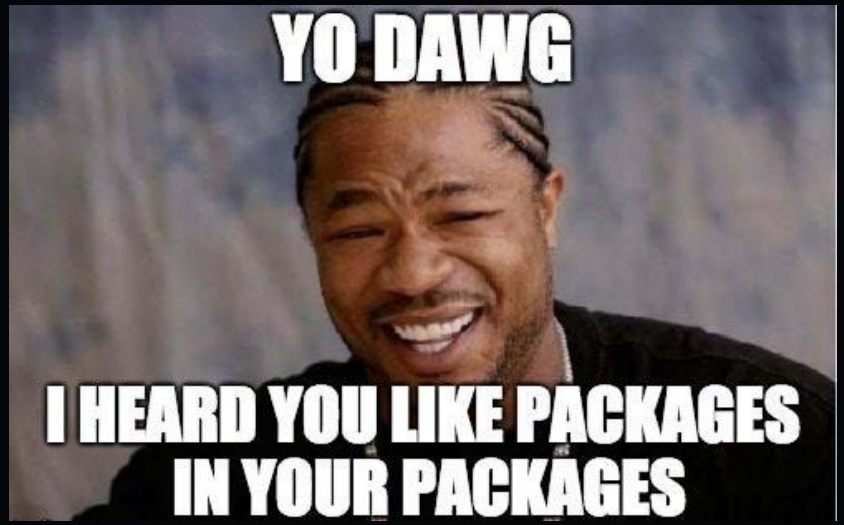


Dozens of tiny apps, each in its own repo, scattered across the galaxy like the surviving Jedi.

One repo again. But many deployables

- Atomic commits across services
- Refactor a shared type. Update every consumer in one PR
- One clone, one review, one history to blame

A monolith is one deployable. A monorepo keeps multiple deployables in one repository. They're not the same thing.



Python made it worse

📁 **src/** vs flat layout

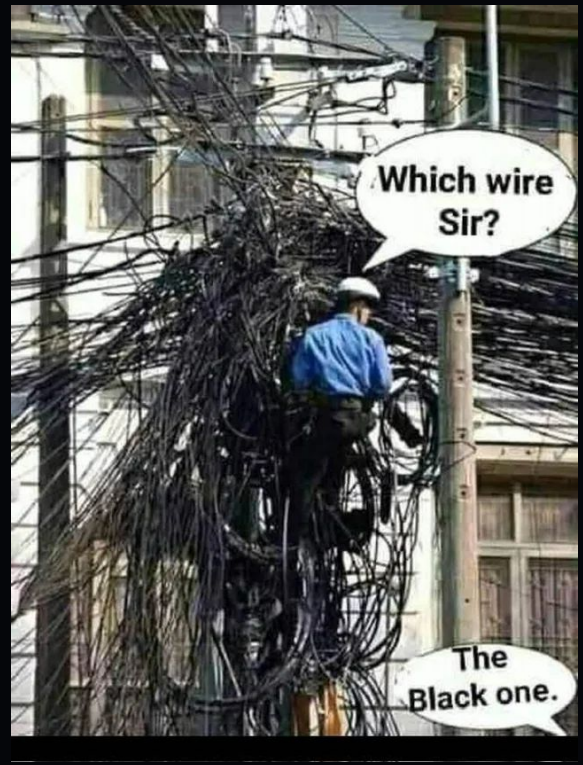
📦 **setup.py** vs **pyproject.toml**

🔗 And the big one: **How do projects share code?**

the answer, for years

```
$ pip install -e ../other-thing
```

A local workaround, not a shared solution. No shared resolution, no recorded dependency, just a path on your laptop. 🪦



// THE THESIS

The folders were never the hard part...

... The wiring between them was

A monorepo is only worth it if the wiring works:

```
one dependency graph · one lockfile · one truth
```

That's the thesis.
Everything that follows puts it to the test.



02

// MOVEMENTS 1-3

The Workshop

Three exercises. One approach: build, troubleshoot, learn and compare.

// HOW THE WORKSHOP HALF WORKS

The loop

01

Build it

Just normal, everyday work.

02

Stumble

Something breaks.
Of course it does.

03

Read & fix

Read the error.
Figure it out.
Fix it.

04

Compare

What would this have cost in a polyrepo?

2.1

// MOVEMENT 1

Meet the band

One workspace, three members, and the toolset that rides on uv's wiring.



One command, every package green

zsh · pizza-monorepo

```
$ git checkout movement-1-start
$ uv sync
Resolved 42 packages in 180ms
$ uv run pytest --cov
Required test coverage of 90.0% reached
===== 39 passed in 2.35s =====
$ uv build --all-packages
Successfully built 3 packages
$ _
```

01

One lockfile

Every package resolves against the same versions

02

Fixed runs

Test the whole project against the same quality gates

03

Build it all

Wheels for every package, straight from the root



The conductor and the band

zsh · pizza-monorepo

```
$ tree -L2
pizza-monorepo-wrkshp/
├── pyproject.toml        🎼 the root: builds nothing, decides everything
├── uv.lock               🔗 one lockfile for the whole band
├── apps/                 🚀 deployables
│   └── pizza_map/           Streamlit map
├── libs/                 🧱 shared
│   └── pizza_shared/        schemas + the PizzeriaSource port
└── services/             📚 business logic
    └── pizza_client/        fixture + HTTP adapters
```

`package = false` *the root ships nothing. It exists to hold the band together*

What vs where

apps/*/pyproject.toml

```toml
# a member pyproject.toml
[project]
name = "pizza-map"
dependencies = ["httpx>=0.28", “pizza-shared”]

[tool.uv.sources]
pizza-shared = { workspace = true }
```

01

dependencies

What your project needs

02

[tool.uv.sources]

Where to find it

Two questions.
One dependency.

Where everything lives

🎼 ROOT

pyproject.toml

`members` glob: defines the band

dev tools: ruff, mypy, deptry, pytest, bump-my-version,

prek, just

shared tool config

constraints and indexes

📦 MEMBER

pyproject.toml

`dependencies`: what it needs

`[tool.uv.sources]`: where to find it

its own version + changelog

release scope + `tests/vars.env`

🧰 RECIPES

.just/

find every workspace member

run shared checks for each member

same commands on your laptop and in CI

The glob is the contract. The recipes use it to give every member the same checks.
uv wires it, just drives it.

What the band plays

01 · apps/

pizza_map

The Streamlit map. The thing we deploy.

02 · services/

pizza_client

Fixture + HTTP adapters for the pizzerias.

03 · libs/

pizza_shared

Schemas + the PizzeriaSource port.

zsh · pizza-monorepo · movement-1-start

```
$ just run
→ http://localhost:8501
```



A new member, for free

zsh · pizza-monorepo

```
$ just new-member services/pizza_stats \
    --depends pizza-shared \
    --description "Aggregations over the rankings"
```

WHAT YOU GOT FOR FREE

- 🏷️ Own version + bump-my-version tag format
- ✨ `{ workspace = true }` already wired
- 📝 CHANGELOG.md + release_scope.env
- 🧪 tests/ with a smoke test and vars.env
- 🎻 The `services/*` glob picked it up. **Nobody edited a list.**

Wire it in: what and where

apps/pizza_map/pyproject.toml

```toml
[project]
dependencies = [ ..., "pizza-stats" ]

[tool.uv.sources]
pizza-stats = { workspace = true }

[tool.deptry]
per_rule_ignores = { DEP003 = [
  "pizza_map", "pizza_client",
  "pizza_shared", "pizza_stats",
] }
```

01

dependencies

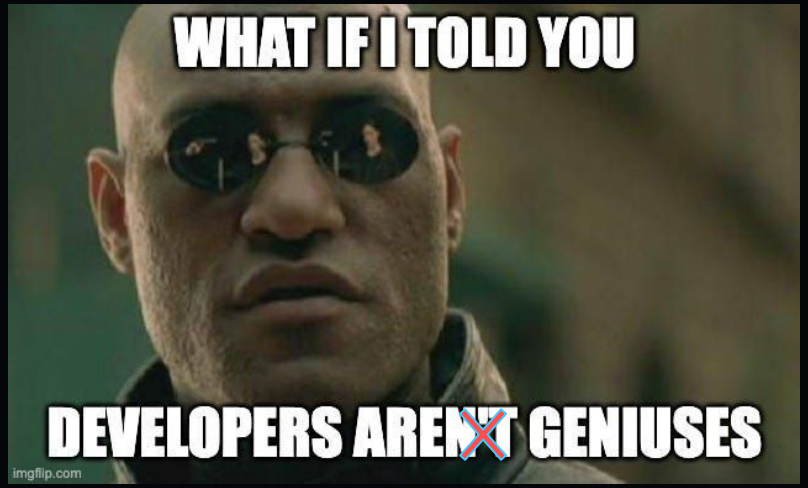
What it needs

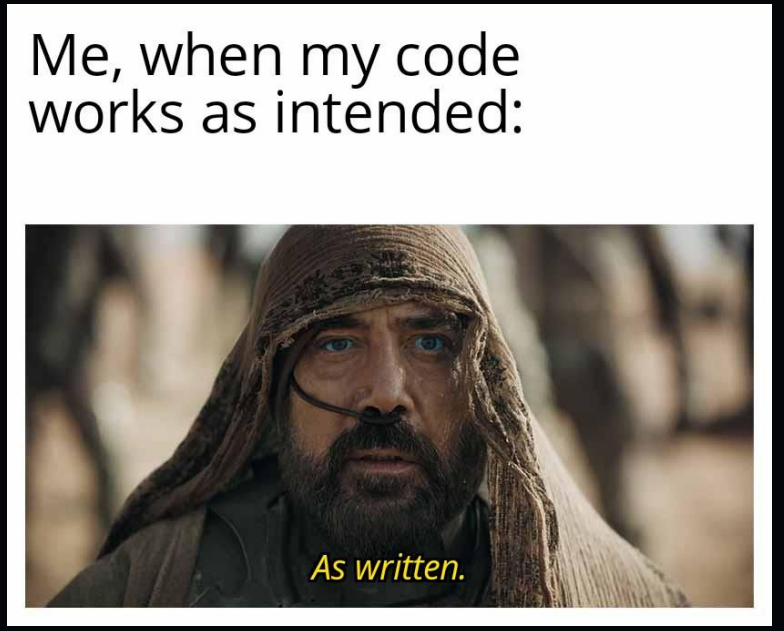
02

[tool.uv.sources]

Where it lives

What and where, again.
`uv sync`:
the lock updates
nothing downloads.





When the lockfile resolves on the first try

```
zsh · pizza-monorepo

$ git checkout movement-2-start  # stash changes first
```

2.2

// MOVEMENT 2

The workspace that argues with you

A version clash, a lying test run, and the gate that catches both.

The code – add the following

shared in Discord

apps/pizza_map/src/pizza_map/views.py

```python
from pizza_stats.stats import by_country

def leaderboard(pizzerias: Iterable[Pizzeria]) -> list[dict[str, object]]:
    """Rows for the country table under the map."""
    return [stat.model_dump() for stat in by_country(pizzerias)]
```

apps/pizza_map/src/pizza_map/app.py

```python
st.subheader("Leaderboard by country")
st.dataframe(
    views.leaderboard(pizzerias), hide_index=True, width="stretch"
)
```

“It worked fine in the old repo”

services/pizza_stats/pyproject.toml

```
dependencies = [
    "pizza-shared",
    "pydantic<2.10",     # 👈 copied from the old service
]
```

Don't sync yet. ✋

Ask the dependency checker first

zsh · pizza-monorepo

```
$ uv run --directory apps/pizza_map --no-sync deptry .
Success! No dependency issues found.
```

Every import declared. Every declaration used.

This is exactly what a polyrepo would check. 😎

zsh · pizza-monorepo · uv sync

```
$ uv sync

error: No solution found when resolving dependencies
       for split (markers: python_full_version >= '3.15'
       and sys_platform == 'win32')

  cause: Because pizza-client depends on pydantic>=2.12
         and pizza-stats depends on pydantic<2.10,
         we can conclude that pizza-client and pizza-stats
         are incompatible.
         And because your workspace requires pizza-client
         and pizza-stats, we can conclude that your
         workspace's requirements are unsatisfiable.
```



Fix, and the polyrepo bill

services/pizza_stats/pyproject.toml

```
# was "pydantic<2.10"
    "pydantic>=2.12",

$ uv sync
$ uv tree --package pizza-stats
```

🌍 **IN A POLYREPO?**

Both repos merge green. Both lockfiles are green. Both ship. They meet for the first time in one image. **At 2am.** 🫠

Here they met at uv sync, before a single commit.

The comfortable lie

zsh · pizza-monorepo

```
$ uv run pytest --cov
41 passed
services/pizza_stats/src/pizza_stats/stats.py   21    0    6    0   100%
TOTAL                                          189    0   20    0   100%
Required test coverage of 90.0% reached.
```

So… done? 🙋

One process runs every member's tests. What can leak?

One member at a time

zsh · pizza-monorepo

```
$ just test
👍 apps/pizza_map   👍 libs/pizza_shared   👍 services/pizza_client
FAIL services/pizza_stats: coverage below 90%
💥 Tests failed for: services/pizza_stats
```

Wait, what? We didn't change a thing...

`just test` runs one pytest per member and measures each member on its own. Same tests, different verdict.



Borrowed coverage 🤦

Coverage records **which lines ran**. Not **which test ran them**.

	`uv run pytest --cov`	`just test`
pytest processes	1, the whole workspace	1 per member
stats.py is run by	the map's test_views.py	pizza_stats' own tests
pizza_stats coverage	**100%**	**below 90%**
Verdict	✅ **green**	✕ **red**

The map's tests have been taking credit for `pizza_stats`.

Add more tests, well done…

zsh · pizza-monorepo

```
# shared in Discord → services/pizza_stats/tests/test_stats.py
$ just test
👍 All tests passed
```

All green. Every member on its own now? 🤔

Separate processes, yes. Now look at the root `pyproject.toml` again. 👀

Borrowed config 🫣

pyproject.toml (root)

```toml
[tool.pytest_env]
env_files = [
  "apps/pizza_map/tests/vars.env",
  "services/pizza_client/tests/vars.env",
  "libs/pizza_shared/tests/vars.env",
]
# TODO: remove… this smells
```

Separate processes, one shared config. The isolation was only half done.



01

Every run

Each member's run still loads all three vars.env files

02

Shared env

The map's PIZZA_SOURCE leaks into the client's tests

03

A smell

A hand-kept list of members. You'll meet it again.

Pull the plug 🔌

zsh · pizza-monorepo

```
# root pyproject.toml: delete [tool.pytest_env] entirely
$ just test
FAILED services/pizza_client/tests/test_factory.py
💥 Tests failed for: services/pizza_client
```

The client never had its own config.

Its tests need PIZZA_SOURCE. The map's vars.env has been setting it for them all along.

All green, for real ✅

💬 ✅ when green

zsh · pizza-monorepo

```
# services/pizza_client/tests/vars.env
PIZZA_SOURCE=fixture
$ just test
👍 apps/pizza_map    👍 libs/pizza_shared
👍 services/pizza_client    👍 services/pizza_stats
👍 All tests passed · every member above 90% coverage
```



📏 **Rule of thumb:** a member path in the root is a promise about **every** member. Sometimes you need it. Know what it hides.

Same tests, split environments

zsh · pizza-monorepo

```
$ just test --no-parallel   # one member after another
$ just test                 # every member at once
```



Own process

Each member gets its own env, its own vars.env, its own coverage.



Parallel is safe

Nothing is shared between runs, so nothing can leak.



Pays off in CI

Heavy suites take as long as the slowest member, not the sum.

What `just test` actually does

🔗 Sync once

`uv sync --locked` up front, then `--no-sync` everywhere

🔀 Per-member runs

One process per member, in parallel

🌱 Own config only

Each member loads only its own `tests/vars.env`

🎯 Scoped coverage

`--cov` measures that member's `src/` only

👻 No tests, no pass

A member without tests is refused. Plain pytest exits green.

📊 Merged for CI

Per-member coverage combined at the end

🎁 `.just/test.just` · it's yours, take it home

The honest trade ⚖️

zsh · pizza-monorepo

```
$ just test
👍 All tests passed
```

🌍 **IN A POLYREPO?**

Honestly? Nothing. You get this for free.

Every repo *is* one project. A monorepo has to **build** it. That's the trade: pay once for the tooling, and you get one lockfile, one graph, and an error at `uv sync` instead of an incident at 2am.

The full quality gate 🚦

zsh · pizza-monorepo

```
$ just check
√ lock  √ lint  √ types  √ deps  √ tests
✗ release scope out of date
  pizza-stats changed
  pizza-map depends on it

# update both scopes, same commit
$ just check-changed
√ all gates green
```

01

Atomic commits

A member and everything that depends on it move in one commit.

02

The gate enforces it

Forget the second half and just check says so.

03

check-changed

Only what changed, plus its dependents.

It works 🎉

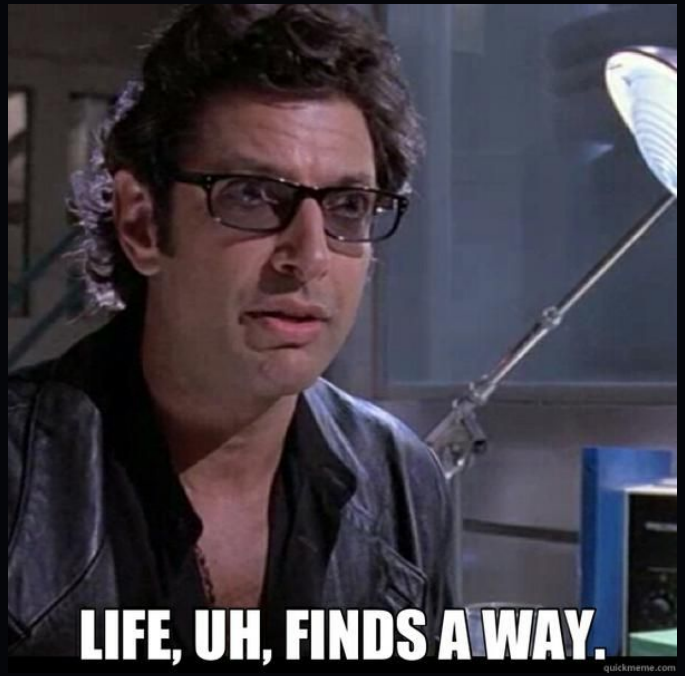
zsh · pizza-monorepo

```
$ just run
```

No reinstall.
No publish.
No version bump.

Editable install: the map saw the change the moment you saved it ✏️

💬 ✅ when you see the leaderboard



2.3

// MOVEMENT 3

These are not the packages you're looking for

Moving a member out, and what quietly moves in.



zsh · pizza-monorepo

```
$ git checkout movement-3-start  # stash changes first
```

Wire in the cow 🐄

💬 🍕 when the pizza cow talks

apps/pizza_map/pyproject.toml

```
dependencies = [ "cowsay>=6.0", ... ]

[tool.uv.sources]
cowsay = { workspace = true }
```

Same two halves again: **what** and **where**. Then one line in `app.py` makes the cow talk.

zsh · pizza-monorepo

```
$ uv sync
+ cowsay==6.0.0 (from file:///.../libs/cowsay)
```

📸 **Remember that line. You'll meet its twin in four minutes.**

Needs change 

A good monorepo makes adding a member cheap.

A great one makes moving it out cheap too.

01

Unwire

`workspace = false` in the app

02

Exclude

The root stops claiming it

03

Delete

When nothing needs it, it goes

`cowsay` really exists on PyPI. Hold that thought. 👀

Two seatbelts

set workspace = false

```
cowsay = { workspace = false }
cause: `cowsay` is included as a workspace member, but does not
       use `workspace = true` in `tool.uv.sources`
```

delete the line entirely

```
cause: `cowsay` is included as a workspace member, but is missing
       an entry in `tool.uv.sources`
       (e.g., `cowsay = { workspace = true }`)
```

While it's a member, uv will not fetch it from anywhere else.

A loud failure is a gift 🎁



Moving out, two lines 🚪

01

The app

workspace = false in the map app

02

The root

Stop the glob from claiming it.

pyproject.toml (root)

```
[tool.uv.workspace]
members = ["apps/*", "libs/*", "services/*"]
exclude = ["libs/cowsay"]

$ uv sync   ✅ green
```

Green. Now look closely at what you just synced. 👀

Look at the diff

💬 🐄 when your cow changed

uv sync · before → after

```
- cowsay==6.0.0 (from file:///.../libs/cowsay)
+ cowsay==6.0
```

6.0.0 became 6.0.

sync ✅ · import ✅ · call ✅

And the cow in your sidebar changed 🐄

The name matched. The index served it.

If that name isn't yours, you are running a stranger's code and nothing failed.

Dependency confusion: Apple, Microsoft and PayPal, 2021.

The silent success is the nightmare.

Then it can go 🗑️

Excluded, the member is still on disk, but outside every gate:

no tests · no checks · no bumps

That's your migration window. Move it to its own repo at your own pace.

zsh · pizza-monorepo

```
$ rm -rf libs/cowsay
$ uv sync    ✅ green
```

The app keeps `cowsay`, now from the index you pinned it to.

Adding took one command. Leaving took three lines.

03

// DEMO

Finale

Versions, gates and CI

Versions move independently

zsh · pizza-monorepo

```
$ just bump services/pizza_client
Bumping services/pizza_client (patch)
  0.1.0 → 0.1.1
Tagged: pizza-client/v0.1.1
```

Only the client moved.

- 🏷️ Own version
- 📝 Own changelog
- 🎯 Own release scope
- 🔖 Own tag

`just changed-members` tells CI what to release.
`just affected-members` adds everything downstream.

// FINALE · SHIP IT

Changed vs affected

change pizza_stats · run just bump



→ is a dependency of

■ changed: own version, changelog, tag

■ affected: depends on a change, bumped too

■ untouched: keeps its version

Change pizza_shared instead, and everything downstream moves. bump-my-version per member, driven by the graph uv already knows.

// FINALE · SHIP IT

Your CI is already written

⚙️ CI TEMPLATES DO

checkout

authenticate

secrets + config

runners + caching

🧰 JUST DOES THE REAL WORK

`just check-changed`

`just test`

`just bump`

versions, changelogs, tags

Same commands on your laptop and in the pipeline.

Green here means green there. ✅

// FINALE · SHIP IT

Everything we've got 🧰

Tool	What it does	Driven by
uv	workspace, one lockfile, sync, build	`just setup` · `just new-member`
just	one entry point, laptop and CI	`just run` · `just check-changed`
ruff	lint + format	`just check`
mypy	types	`just check`
deptry	imports vs declared deps	`just check`
pytest + cov	tests, per-member coverage	`just test` · `just test-member`
bump-my-version	versions, changelogs, tags	`just bump` · `changed / affected-members`
uv audit	known CVEs, and who pulls them in	`just audit-uv` · `just audit-blame`
docker compose	the whole system, locally	`just compose`

FIN

// WHAT ELSE

Wrap

More...

What we didn’t have time for

- One **`requires-python`** for the whole workspace
- **`uv lock --upgrade-package pydantic`** · one upgrade, every member, one PR
- **`--package`** subsets · why the image stays small
- **`exclude-newer`** · every laptop resolves the same snapshot or PyPI
- **`uv audit`** on a schedule · one dashboard for the whole domain

> *It’s all in the repo. And in the articles.*

When not to do this

- Team of 2, one deployable → a modular monolith is probably enough
- Members that genuinely need incompatible versions
- Hard ownership or access boundaries between teams
- Nobody willing to own the tooling → It is small, but someone has to

> *"Does this repo map to one domain that deploys and evolves together?"*

In one breath

MONOREPO

- 🕸️ One graph
- 🔗 one lockfile
- 🧪 per-member tests
- 🏷️ per-member versions + tags
- 🏆 one quality gate
- 🔐 one security gate

The point was never the tools. It was the trust they compound.

🗣️ Discord stays open for questions

The repo is your:

- `main` branch is a full template
- `.just/` is meant to be used too

Thank you

Questions?

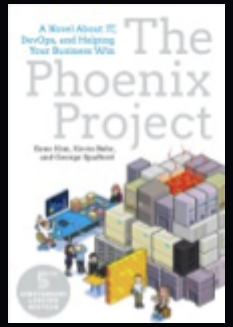
CONNECT

Connect you must

ARTICLES

Read you will

FOR FUN



~

> find more about me
> on dervis.dev



MAY THE SOURCE BE WITH YOU...